**Invariably Abandoned**

# Why you shouldn't use offset/limit (visibly)

**DanFromDownUnder**
04 Feb 2026 • 5 min read

Pagination using offset/limit is ubiquitous, simple to understand and works excellently in most cases... but you shouldn't use it, at least not ***visibly***.

This article will hopefully convince you of the shortfalls of offset/limit and equip you with some strategies to avoid them.

## The offset/limit API

We're going to use an example of a JS project calling a rest api example for demonstration, but it's easy the transfer the same idea to GraphQL or RPC etc and other languages.

Querying using offset/limit might look something like this:

```
const results = [];
for (let pageNumber = 0; pageNumber < PAGES_TO_FETCH; pageNumber++) {
    const offset = pageNumber * PAGE_SIZE;
    const requestUrl = `${BASE_URL}pokemon?offset=${offset}&limit=${PAGE_S
    console.log(`Fetching ${requestUrl}`);
    const response = await fetch(requestUrl);
    const resultWrapper = await response.json();
    results.push(...resultWrapper.results);
}
```

This gives the caller a huge amount of flexibility, including being able to change the page size or jump to any page (more on that layer).


When you see UI like this, it's almost certainly an offset/limit implementation (source: https://www.npmjs.com/package/react-paginate)

## How offset/limit works

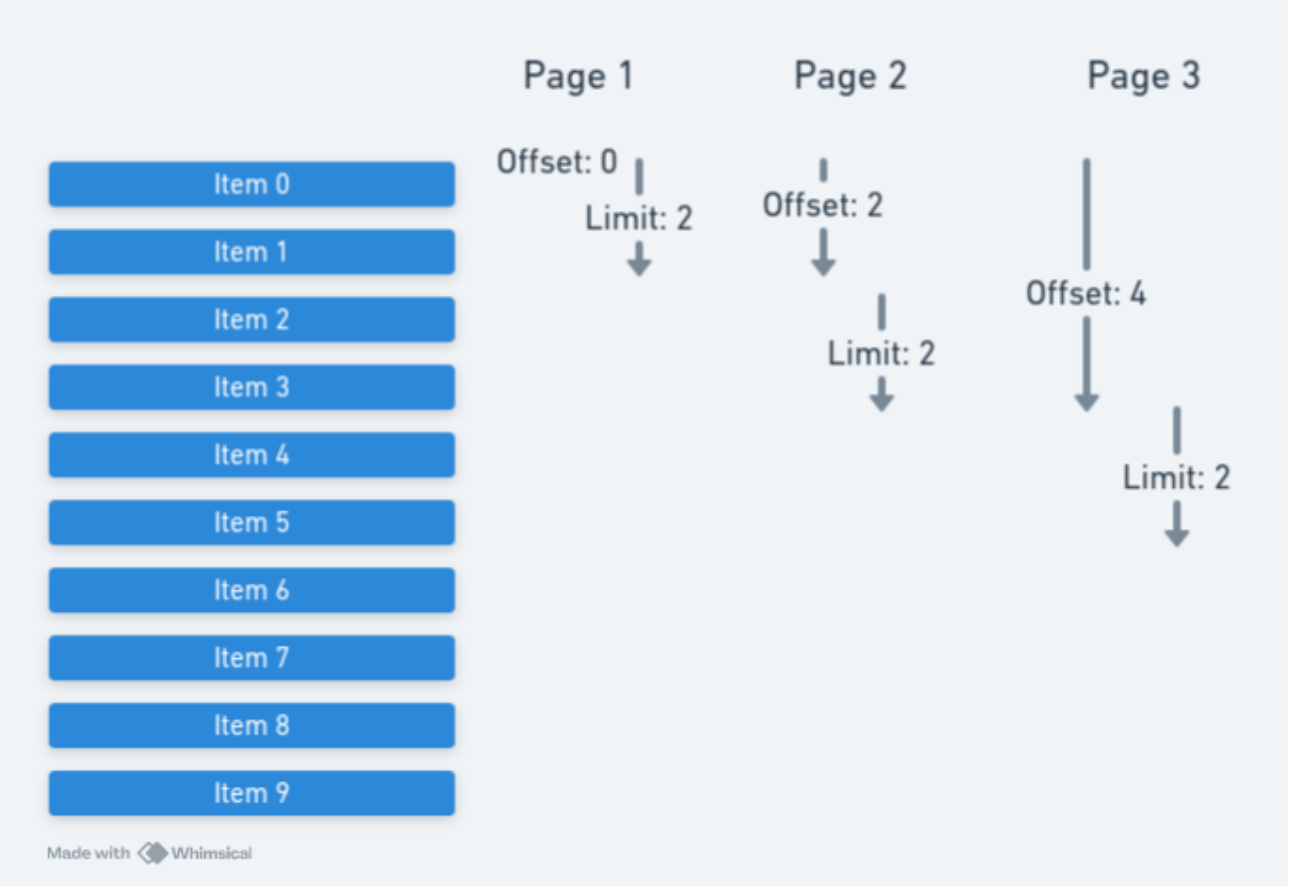
Likely you need no explanation of how offset limit works, you simply provide two integers, the first being a starting point and the second being the number of rows to be returned.


If you have 10 items numbered 0 to 9 and you provide an `offset` of `2` and a `limit` of `3` you will receive items 2 to 5.

What's interesting to discuss is how a database will go about getting these three results: It will be done in two parts, first a scan of `offset` rows to get to the starting point, and then a scan of `limit` rows to collect the actual results. As you can imagine in the above example any database system will perform these small scans almost instantly.

We can fairly intuitively reason about how this will scale as well: even in the cases of very large data sets the first few pages will be returned almost instantly as the number of rows that need to be scanned will be very low. However the last page for example, will require scanning over the entire data set.

An oddity we can already notice is for each page the same data will be scanned through the same data again and again.




For every page we scan over the items of the pages before. The offset scans get longer for each subsequent page, the limit scans remain stable.

## Some Performance Numbers

In order to demonstrate the performance of an offset/limit approach, a database can be set up with dummy data. PostgreSQL is used in this case however all engines will look relatively similar. This demonstration uses a data set size of 1 million rows to simulate a large (but still reasonable) data set.

The database instance used for testing is running on a dell xps laptop and is not under any load, the only queries being run against it are these test queries.

```
create table sample_dataset (
  id serial primary key,
  name varchar not null,
  created_at timestamp not null
);

insert into sample_dataset (name, created_at)
  select
    md5(random()::text) as name,
    timestamp '2000-01-01 00:00:00' +
       random() * (timestamp '2025-01-01 00:00:00' -
                   timestamp '2000-01-01 00:00:00') as created_at
  from generate_series(1,1000000) i;
```

Create a table with dummy data

We can now run through a few scenarios and measure the rough performance. With our million tows and using a reasonable page size of 1000 items we have 1000 pages to play with.

```
-- Page 1

select *
from sample_dataset
offset 0
limit 1000

-- 1000 row(s) fetched - 0.002s

-- Page 2

select *
from sample_dataset
offset 1000
limit 1000

-- 1000 row(s) fetched - 0.002s
```

As expected, the first two pages come back instantly

```
-- Page 500

select *
from sample_dataset
offset 500000
limit 1000

-- 1000 row(s) fetched - 0.014s
```

For the middle page we're still getting results back in a small fraction of a second, but it's orders of magnitude slower (7x) than the first pages

```
-- Page 999

select *
from sample_dataset
offset 999000
limit 1000

-- 1000 row(s) fetched - 0.027s
```

For the last page of the data set we can see the amount of time taken is scaling pretty linearly

To confirm our reasoning above about the scanning we can run an explain on the page 999 query. We get back the following:

```
Limit  (cost=18315.67..18334.00 rows=1000 width=37)
  →  Seq Scan on sample_dataset  (cost=0.00..18334.00 rows=1000000 width=
```

Sure enough we're simply scanning 1 million rows to get the results.

Let's put a more "worst-case-scenario" data set to the test. Using the same setup as above but ramping up to 100 million rows.

```
-- Page 1

select *
from large_sample_dataset
offset 0
limit 1000

-- 1000 row(s) fetched - 0.002s

-- Page 99999

select *
from large_sample_dataset
offset 99900000
limit 1000

-- 1000 row(s) fetched - 3s
```

We're now looking at 3s query times getting the last page on the same unloaded PostgreSQL instance with plenty of grunt.

## Initial Performance Takeaways

We can take a few things away from our testing

### Postgres can scan really fast

We just scanned one million rows in less than a tenth of a second. In that example we probably don't need to do better than offset/limit.

However we have to imagine that if thousands of users and making these queries concurrently we would see much worse reponse times.

We've also seen that if the data sets are even larger we start to see unreasonable response times.

## Offset/limit scales with O(n) complexity

As expected, the more rows we needed to scan, the longer the query took.

## The contender: cursors

The main alternative

## Resources

**Pagination | GraphQL**

GraphQL


**JSON API — “Cursor Pagination” Profile**

{j}


## React Single Page Applications & Docker: An unlikely match

Ready to go You've created your perfect single-page application, you have your bundler all configured (whether that be vite, create-react-app or some new kid on the block) and you're ready to make it available to the world. Should you deploy to your favourite object storage...

04 Feb 2026 5 min read